

Arsham Ardeshirian

arsham202@gmail.com | (437) 263-1827 | TORONTO, ON |

Github.com/arsham909 | LinkedIn.com/in/arshamardeshirian/ | Eligible to work in Canada

SUMMARY

Software engineer with hands-on experience building production Python backends using Django, FastAPI, PostgreSQL, and AWS. AWS Certified Solutions Architect – Associate. Shipped a full-stack repair management platform that cut data-entry errors 40% and improved operational efficiency 25%. Comfortable owning features end-to-end: REST API design, async processing (Celery/Redis), containerization (Docker), and infrastructure-as-code (AWS CDK).

TECHNICAL SKILLS

Languages: Python, Java, JavaScript, SQL, Bash

Frameworks: Django, Django REST framework, FastAPI, Flask, Streamlit, HTMX

Databases: PostgreSQL, Redis, DuckDB, SQLite, DynamoDB

Cloud & DevOps: AWS (Kinesis, Lambda, S3, DynamoDB, SNS, Bedrock, CDK, IAM), Docker, Docker Compose, Git, GitHub Actions, Linux, CI/CD

Testing: pytest, unit testing, integration testing, TDD

Libraries: OpenCV, Pandas, NumPy, Scikit-learn, Pydantic, Pandas, Celery

Concepts: REST APIs, Asyncio / async programming, Event-driven architecture, Microservices, Object-Oriented Programming (OOP), SQL query optimization, Agile

Certifications: AWS Certified Solutions Architect – Associate (March 2026)

WORK EXPERIENCE

Automation Systems Developer | Accu Electric LLC | Aug. 2025 – Mar 2026

[github](#)

Tech: Django, PostgreSQL, HTMX, HTML/CSS, Redis, Celery, FastAPI, Docker

- Engineered an internal repair-tracking platform(Django, PostgreSQL, Docker) used by 8+ technicians daily, replacing **3 paper-based workflows** and **reducing manual data-entry errors by 40%**.
- Built Streamlit dashboards backed by Django REST Framework APIs over PostgreSQL, automating KPI reporting for 3 departments and reducing report generation time from 4+ hours to real-time.
- cut average job turnaround by 2-3 hours (≈25% throughput gain) by implementing async background task processing with Celery + Redis and digitizing technician workflows.
- Wrote unit and integration tests with pytest, containerized the stack with Docker Compose, and deployed to a Linux VPS behind Nginx — owning the full SDLC from requirement to production.

PROJECTS

Log Sentinel | [github](#)

Tech: AWS Kinesis · Lambda · S3 · Bedrock · SNS · DynamoDB · CDK · Python · Docker

- Developed a real-time log ingestion pipeline on AWS, streaming and processing application logs through Kinesis and Lambda with sub-second latency, serving as the backbone for AI-powered anomaly detection.
- Integrated Amazon Bedrock to automatically classify anomaly severity and trigger targeted SNS alerts, replacing manual log review with automated triage and surfacing incidents via a live Streamlit monitoring dashboard.

- Deployed production-ready infrastructure as code with AWS CDK (Python) and replicated the full cloud environment locally using Docker Compose and LocalStack, reducing environment setup time to under 5 minutes.

Redis Server from Scratch in Python | [github](#)

Tech: Python, Asyncio, RESP protocol, TCP, event loops

- Architected a single-threaded asynchronous event loop to serve many concurrent client connections simultaneously, mirroring real Redis architecture rather than relying on OS threads.
- Implemented 15 Redis commands and millisecond-level key expiry (PX option), backed by an in-memory key-value store with TTL eviction and sub-millisecond lookup time.
- Extended the server to support list operations (RPUSH, LPUSH, LPOP), deepening the command dispatch layer to match production-grade Redis protocol structure.

Portfolio Tracker | [github](#)

Tech: Python · DuckDB · Typer · yfinance · Pydantic · pytest

- CLI stock portfolio manager with real-time Yahoo Finance data, DuckDB analytics, and unrealized P&L calculation via a SQL view, supporting Monte Carlo simulation for portfolio risk analysis.
- Layered architecture built with TDD: unit tests mock the DB layer, integration tests run against a real database, achieving 100% test coverage.

HTTP Server from Scratch in Python | [github](#)

Tech: Python, TCP sockets, threading, HTTP/1.1

- Constructed a concurrent HTTP/1.1 server using only the Python standard library, completing the full CodeCrafters HTTP server track including all extension stages.
- Added concurrent client handling via a thread-per-connection model, enabling the server to serve multiple simultaneous clients without blocking.
- Authored raw TCP socket handling, an HTTP/1.1 request parser (request line, headers, body), routing, response serialization, gzip compression via Accept-Encoding, and GET/POST with path-param + file-upload endpoints – zero external dependencies .

EDUCATION

Bachelor of Mechanical Engineering – University of Kharazmi, 2022